Setting correct memory parameters

MySQL uses temporary tables a lot when processing complex queries that involve joins and sorting. The default size of a temporary table is very small and is not sufficient for bigger data sets. To set a temporary table size, add the following to your *my.cnf*:

```
tmp-table-size = 1G
max-heap-table-size = 1G
```

It is important to note that setting larger values for temporary tables means more RAM will be consumed. If the system does not have sufficient RAM, then the values should not be increased. *Mysqltuner* can provide information on whether the current settings of the server will overrun the available RAM or not. A good RAM usage figure is 80-90 per cent. Beyond 90 per cent implies operating on the borderline and can cause failure, assuming the database is the only service on the server. If the same machine is used for other services such as the Web, then that needs to be taken care of as well. However, it is a bad idea to run the Web and database on the same server in large workloads.

Buffer sizes

Two important entities that can be tuned are often missed, namely, join buffer size and sort buffer size. These buffers are allocated per connection and play a significant role in the performance of the system. Join buffer is used, as the name suggests, to process joins – but only full joins on which no keys are possible.

Sort buffer size is used to sort data. If your application involves a lot of sorting, then this should be increased. The system status variable *sort_merge_passes* will tell you whether the value needs to be increased or not. This variable should be as low as possible.

InnoDB settings

As stated earlier, the more the RAM, the better the performance. The performance of an InnoDB system is directly dependent on the size of the data that is present and the buffer pool size. A thumb rule is that the size of the buffer pool should be at least as large as the data that is being stored – this is not feasible in all cases and not required either, since not all the data present in the database may be required at the same time. But the buffer pool size should be large enough to accommodate data that is worked upon frequently.

The InnoDB log file is used for storing a redo log, which can be replayed in case of a power failure or database crash. If there are a lot of writes in the application, then the size of the log file must be increased. The optimal size of the log file can be calculated by observing the status variable *innodb_os_log_written*. It is a counter (bytes) which gets incremented for every byte written to the log file. So the difference between the values of the variable in a given time period (60 seconds) gives the optimal log file size to store exactly one minute worth of recovery data. To store longer periods of redo log, the size can be a multiple of the unit time interval. The status variable *innodb_log_waits* should be checked as well if the log size really needs tuning. In many set-ups the log size is unnecessarily large, which increases the time required for recovery when there is a crash or power failure.

Most of the servers these days have multiple CPUs (physically, or as SMP). If the InnoDB buffer pool size is more than a gigabyte then it can be divided across the number of CPUs using the setting *innodb-buffer-pool-instances*. The ideal value for the number of buffer pool instances is the number of CPUs you have on the server, and it is important to note that each instance must be at least 1 GB.

File system tuning

The file system on which database data is stored is an important aspect of performance tuning. Most Linux servers use *ext4* or *xfs* but without one important setting being enabled – *noatime*. Whenever a file is read, the access time of the file is updated. This is of little utility and disabling it will yield a lot of performance benefit. Additionally, it is possible to use a raw disk as storage in case of InnoDB – the whole InnoDB data file can be stored over a raw disk partition. This mode is good if you have a dedicated disk and can help to avoid double buffering by the file system as well as MySQL.

As of Ubuntu 16.04, it is possible to run ZFS on Linux. Linux file systems by default use the LRU (Least Recently Used) algorithm for file system page caching – the oldest page is evicted when memory is needed. In contrast, ZFS has ARC – Adaptive Replacement Cache, which uses a page replacement algorithm that takes into account both recently read blocks as well as frequently read blocks— basically a combination of LRU and LFU (Least Frequently Used). This does help to achieve some performance benefit, but it depends on the workload. In addition to ARC, it also supports L2ARC – an ARC that can be stored on a memory device which is slightly slower than RAM, such as NVMe or SSD. Storing InnoDB with default ZFS settings will not provide optimal performance – the record size of InnoDB storage should be 16k and that of InnoDB log storage should be 128k. In case of the *innodb-file-per-table* flag (which is 'on' by default), the InnoDB data files are created inside individual database directories instead of being stored in a single large file. The whole MySQL dataset needs to have a record size of 16k in that case.

This covers relatively a large part about MySQL and MariaDB performance tuning; the rest depends on the application workload. I have successfully configured servers for 1k to 5.5k queries per second on an average, using these strategies. END

By: Nilesh Govindrajan

The author is a software engineer based in Pune who loves to work on server optimisation. He can be contacted at *https://nileshgr.com* or *@nileshgr* on Twitter.